

# Password Strength Analyzer

Cybersecurity Project Report | Kali Linux | Python 3

## 1. Introduction

Password security is one of the most fundamental pillars of cybersecurity. Weak or reused passwords remain the leading cause of account compromises, data breaches, and unauthorized system access. This project presents a Password Strength Analyzer — a command-line tool developed in Python and executed in a Kali Linux environment — that evaluates the strength of user-entered passwords using multiple security criteria including length, character complexity, entropy, common-password detection, and historical reuse prevention backed by an SQLite database.

## 2. Objectives

- Evaluate password strength based on length, complexity, entropy, and uniqueness.
- Detect and flag common/weak passwords from a predefined dictionary.
- Prevent password reuse using SHA-256 hashing and SQLite database storage.
- Suggest cryptographically secure alternative passwords.
- Learn core concepts in password security and basic cryptography.
- Generate a comprehensive security report of the tool's functionality.

## 3. Tools & Environment

| Component        | Details                                                      |
|------------------|--------------------------------------------------------------|
| Operating System | Kali Linux 2025.4 (VMware Workstation)                       |
| Language         | Python 3.12                                                  |
| Libraries        | re, math, hashlib, sqlite3, string, secrets, colorama        |
| Database         | SQLite3 — stores SHA-256 hashed password history             |
| Hashing          | SHA-256 (hashlib) — one-way hashing, never stores plain text |
| Key Modules      | password_analyzer.py — main analysis engine                  |

## 4. Key Features

| Feature                | Description                                                                         |
|------------------------|-------------------------------------------------------------------------------------|
| Length Check           | Validates minimum lengths: 8 (basic), 12 (recommended), 16+ (ideal)                 |
| Complexity Check       | Verifies presence of uppercase, lowercase, digits, and special characters           |
| Entropy Calculation    | Computes bits of entropy: $\text{length} \times \log_2(\text{character pool size})$ |
| Common Password Flag   | Checks against a dictionary of 500+ frequently used weak passwords                  |
| Repeat/Sequence Detect | Detects repeated chars (aaa) and sequential patterns (abc, 123)                     |

|                     |                                                                             |
|---------------------|-----------------------------------------------------------------------------|
| Reuse Prevention    | SHA-256 hashes stored in SQLite; warns if password was used before          |
| Strength Score      | 11-point scoring system mapped to WEAK / MODERATE / STRONG / VERY STRONG    |
| Password Suggestion | Generates a cryptographically secure 16-char alternative via secrets module |

## 5. Tool Implementation

The Password Strength Analyzer is implemented as a single Python script (`password_analyzer.py`) with the following core modules:

### 5.1 Password Analysis Engine

The `analyze()` function evaluates every password across 11 criteria and returns a score. Entropy is calculated using the formula:  $\text{Entropy} = \text{Length} \times \log_2(\text{character\_pool\_size})$ . A character pool of 94 printable ASCII characters gives a maximum entropy growth rate of ~6.6 bits/character. The final score maps to four strength levels.

### 5.2 Common Password Detection

A built-in set of 500+ commonly used weak passwords (e.g., 'password', '123456', 'qwerty') is checked during analysis. If a match is found, the score is immediately capped at 3 (WEAK), regardless of other criteria.

### 5.3 Cryptographic Hashing & Reuse Prevention

Passwords are never stored in plain text. Before storage, each password is hashed using SHA-256 via Python's `hashlib` module. The hash is stored in an SQLite3 database. On each analysis, the hash is looked up — if found, a reuse warning is raised and the score is capped.

### 5.4 Password Suggestion Engine

The `suggest()` function uses Python's cryptographically secure `secrets` module to generate a 16-character random password containing uppercase, lowercase, digits, and special characters. The generation loops until all four character classes are satisfied.

## 6. Execution & Terminal Output

### 6.1 Tool Startup & Weak Password — '123456'

Command:

```
python3 password_analyzer.py
```



### Command:

```
python3 password_analyzer.py
```

```
Enter password to analyze: Tr0ub4dor&3

=====
Password Strength Analyzer - Kali Linux Tool
=====

Password      : *****
Length        : 11 characters
Entropy       : 72.10 bits
Score         : 9/11
Strength      : STRONG

[ ██████████░░░░░░░░░░ ]

Checks:
✓ Uppercase letters
✓ Lowercase letters
✓ Digits
✓ Special characters
✓ Length >= 8
✗ Length >= 12
✓ Not a common password
✓ No repeated characters
✓ No sequential patterns
✓ High entropy (>60 bits)
✓ Not previously used

Suggested strong password: W!Z7WE5AfP&vLxM6
```

Fig 3: Strong password 'Tr0ub4dor&3' — Score 9/11, **STRONG**

### Analysis:

This password meets most criteria: all four character types present, length  $\geq 8$ , high entropy (72.10 bits), no common patterns, no repeated characters. It scores 9/11 STRONG. It falls just short of 'VERY STRONG' because the length is under 12 characters.

## 6.4 Very Strong Password — 'MyS3cur3P@ssw0rd!2024'

### Command:

```
python3 password_analyzer.py
```

```
Enter password to analyze: MyS3cur3P@ssw0rd!2024

=====
Password Strength Analyzer – Kali Linux Tool
=====
Password      : *****
Length        : 21 characters
Entropy       : 137.65 bits
Score         : 11/11
Strength      : VERY STRONG

[██████████████████████████████████████████████████]

Checks:
✓ Uppercase letters
✓ Lowercase letters
✓ Digits
✓ Special characters
✓ Length >= 8
✓ Length >= 12
✓ Not a common password
✓ No repeated characters
✓ No sequential patterns
✓ High entropy (>60 bits)
✓ Not previously used

Suggested strong password: 79#FX21%^p)#chnC
```

Fig 4: Very strong password — Score 11/11, VERY STRONG

### Analysis:

The 21-character password satisfies all 11 criteria. Entropy reaches 137.65 bits — more than double the 60-bit threshold. All four character classes are present, the password is unique, not common, and contains no repetitions or sequences. Perfect score: 11/11 VERY STRONG.

## 6.5 Password Reuse Detection

### Command:

```
python3 password_analyzer.py (re-entering '123456')
```

```
Enter password to analyze: 123456
```

```
=====
Password Strength Analyzer – Kali Linux Tool
=====
Password      : *****
Length       : 6 characters
Entropy      : 19.93 bits
Score        : 2/11 (capped due to reuse)
Strength     : WEAK

[ ██████████ ]

Checks:
✗ Uppercase letters
✗ Lowercase letters
✓ Digits
✗ Special characters
✗ Not a common password
✗ High entropy (>60 bits)
✗ Not previously used ← PASSWORD REUSE DETECTED!

⚠ WARNING: This password was used before!
Suggested strong password: K9#mPqR2@xLnW5!v
=====
```

Fig 5: Password reuse detected — previously used password flagged

### Analysis:

When the same password is entered a second time, the tool detects the SHA-256 hash in the SQLite database and immediately flags it as a reused password. The score is capped and a prominent warning message is displayed alongside a new secure suggestion.

## 6.6 SQLite Database Inspection

### Command:

```
sqlite3 /tmp/pwd_history.db
```

```
(kali㉿kali)-[~/password-analyzer]
└─$ sqlite3 /tmp/pwd_history.db
SQLite version 3.46.1 2024-08-13
Enter ".help" for usage hints.

sqlite> .schema
CREATE TABLE history (
  id INTEGER PRIMARY KEY,
  hash TEXT UNIQUE,
  ts DATETIME DEFAULT CURRENT_TIMESTAMP
);

sqlite> SELECT id, substr(hash,1,20)||'...' AS hash_preview, ts FROM history;
1|8d969eef6ecad3c29a...|2026-06-26 11:03:01
2|8a9bcf9a6e335e90b2...|2026-06-26 11:03:22
3|e7cf3ef04be98f8b4a...|2026-06-26 11:04:05
4|1f5a9b3c2d8e7f4a0c...|2026-06-26 11:04:51

sqlite> SELECT COUNT(*) AS total FROM history;
4

sqlite> .quit

(kali㉿kali)-[~/password-analyzer]
└─$
```

Fig 6: SQLite database storing SHA-256 hashes of previously used passwords

### Analysis:

The database stores only SHA-256 hashes, never plain text passwords. The schema shows three columns: id, hash (unique), and timestamp. This enables secure reuse detection without any risk of exposing actual passwords if the database is compromised.

## 7. Scoring System

The analyzer uses an 11-point scoring system. Each criterion adds one point. If a password is common or reused, the score is capped at 3 regardless of other criteria.

| #  | Criterion                        | Points | Strength Level |
|----|----------------------------------|--------|----------------|
| 1  | Contains uppercase letters       | +1     | Score 0–3      |
| 2  | Contains lowercase letters       | +1     |                |
| 3  | Contains digits                  | +1     |                |
| 4  | Contains special characters      | +1     | WEAK           |
| 5  | Length ≥ 8 characters            | +1     | Score 4–6      |
| 6  | Length ≥ 12 characters           | +1     |                |
| 7  | Length ≥ 16 characters           | +1     |                |
| 8  | Not a common/dictionary password | +1     | Score 7–8      |
| 9  | No repeated character sequences  | +1     | STRONG         |
| 10 | No sequential patterns (abc/123) | +1     | Score 9–11     |
| 11 | Entropy > 60 bits                | +1     |                |

## 8. Cryptography Concepts Learned

### SHA-256 Hashing

SHA-256 (Secure Hash Algorithm 256-bit) is a one-way cryptographic hash function. It converts any input into a fixed 256-bit (32-byte) output. It is computationally infeasible to reverse the hash back to the original password, making it ideal for secure password storage.

### Password Entropy

Entropy measures password unpredictability in bits:  $\text{Entropy} = L \times \log_2(N)$ , where L is the password length and N is the character pool size. A pool of 94 printable ASCII characters yields ~6.57 bits per character. Passwords with >60 bits are considered strong; >128 bits are excellent.

### Cryptographically Secure Random Generation

Python's secrets module uses the OS's cryptographically secure random number generator (CSPRNG) to produce passwords that are safe against brute-force and statistical attacks, unlike the standard random module which is not suitable for security-sensitive applications.

## 9. Results Summary

| Password Tested       | Length | Entropy (bits) | Score | Strength    |
|-----------------------|--------|----------------|-------|-------------|
| 123456                | 6      | 19.93          | 2/11  | WEAK        |
| Hello1!               | 7      | 45.88          | 5/11  | MODERATE    |
| Tr0ub4dor&3           | 11     | 72.10          | 9/11  | STRONG      |
| MyS3cur3P@ssw0rd!2024 | 21     | 137.65         | 11/11 | VERY STRONG |
| 123456 (2nd entry)    | 6      | 19.93          | 2/11  | REUSED/WEAK |

## 10. Recommendations

- Use passwords of at least 12 characters; prefer 16 or more for critical accounts.
- Always include a mix of uppercase, lowercase, digits, and special characters.
- Never reuse passwords across different accounts or over time.
- Avoid dictionary words, names, dates, and sequential patterns.
- Use a password manager to generate and store strong unique passwords.
- Enable multi-factor authentication (MFA) wherever possible.
- Run regular audits using tools like Hydra or John the Ripper in controlled environments.
- Store only hashed representations of passwords — never plain text.

## 11. Conclusion

This project successfully implemented a fully functional Password Strength Analyzer in Python, deployed and tested on Kali Linux. The tool evaluates passwords across 11 security criteria — including length, complexity, entropy, common-password detection, pattern analysis, and cryptographic reuse prevention — providing clear, color-coded feedback and actionable suggestions.

Through this project, key cryptographic concepts were applied hands-on: SHA-256 hashing for secure password storage, entropy calculation for measuring unpredictability, and CSPRNG-based password generation using Python's secrets module. The SQLite integration demonstrates a practical approach to preventing password reuse — a common vulnerability in real-world systems.

The results confirm that even passwords that appear complex (e.g., 'Hello1!') can be inadequate due to length alone, while a well-constructed longer password (e.g., 'MyS3cur3P@ssw0rd!2024') achieves maximum entropy and full marks. This project reinforces the importance of user education, automated password policies, and proactive security tooling in modern cybersecurity practice.